

LECTURE 8

Transformers, Intuitively

An analogy-first walk-through: meaning as coordinates, words that change with context, attention as a radio net, and how the encoder and decoder share the work

AK DHAMIJA, Researcher & Instructor

AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Learning Objectives & Session Structure

By the end of this lecture you will be able to

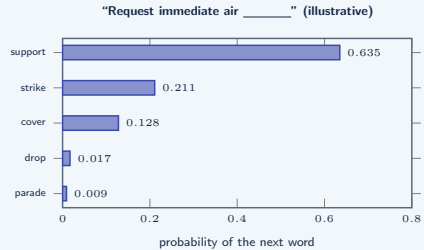
1. Explain what a language model does, and how it writes one token at a time
2. Describe embeddings as coordinates on a "meaning map"
3. Explain why a fixed vector per word fails, and what a contextual vector is
4. Describe the jobs of the encoder and the decoder
5. Explain attention using queries, keys and values — and compute one by hand
6. Explain multi-head attention, residual connections, normalisation and the feed-forward layer
7. Explain cross-attention in an encoder–decoder model

Session Structure (45 min)

- **5 min** — Opening Q&A: CNNs (L7)
- **10 min** — Language models; meaning as numbers; the "battery" problem
- **6 min** — Encoder, decoder, tokens, positions
- **14 min** — Attention, multi-head, the full block, the decoder
- **10 min** — Auto-quiz (15 questions)

How this lecture fits: it builds intuition. Lecture 9, *Transformers & Attention*, gives the full mathematics and the Vaswani et al. architecture block by block.

What a Language Model Actually Does



Scores $(3.1,\ 2.0,\ 1.5,\ -0.5,\ -1.2)$ turned into probabilities by softmax.

One job: predict the next token

Given the words so far, a language model outputs a probability for every token in its vocabulary. "Parade" is possible but unlikely; "support" fits what the model has seen in thousands of similar sentences.

Writing a whole answer = repeating the job

Request immediate air **support**

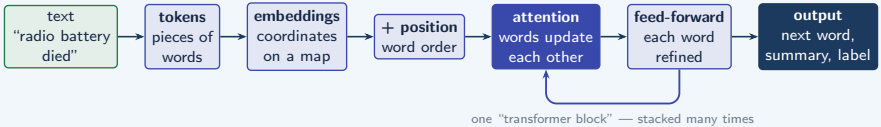
Request immediate air support **at**

Request immediate air support at **grid**

. . . each new word is appended and fed back in

A chat assistant's long, fluent reply is this loop run hundreds of times. "Large" language models do the same job with billions of learned parameters and vast training text.

The Journey Through This Lecture



Running example

The word **battery**: an *artillery* battery or a *radio* battery? We will watch a transformer work it out from context.

Analogies used

Grid references (embeddings), a radio net (attention), a sensor suite (multi-head), margin notes (residuals).

Analogies build intuition but are not the mechanism. Each slide also shows what is *actually* computed.

Why a Simple ID Number Is Not Enough

Computers need numbers

A model cannot read letters. The simplest idea is to number every word in a dictionary:

word	ID
howitzer	4 118
hospital	4 119
howl	4 120

IDs follow spelling, not meaning: *howitzer* sits next to *hospital*, and nothing in "4 118" says it is a weapon. Arithmetic on IDs is meaningless.

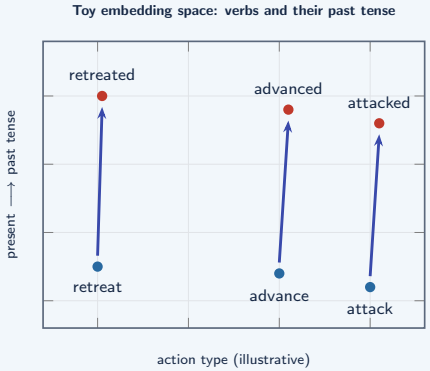
Better: give each word a position on a map

A military grid reference places a location by several coordinates; nearby references are nearby places. An **embedding** places a word by a list of numbers; nearby lists are words with similar meaning.



a 2-D slice of a meaning map (illustrative)

Directions on the Meaning Map Carry Meaning



A "past-tense" direction

In a well-trained space, the step from *advance* to *advanced* points roughly the same way as *retreat*→*retreated* and *attack*→*attacked*. Grammar has become geometry.

VECTOR ARITHMETIC (TOY NUMBERS)

$$\overrightarrow{\text{retreat}} - \overrightarrow{\text{advance}} + \overrightarrow{\text{advanced}}$$
$$= (-1.0, 0.25) - (1.0, 0.2) + (1.1, 1.4) = (-0.9, 1.45)$$

nearest word: **retreated** $(-0.95, 1.5)$

Other directions found in real embeddings: singular→plural, country→capital, unit→parent formation. Lecture 9 works the classic male→female example in full.

Where Embeddings Come From

Nobody writes them by hand

The numbers start random. The model reads enormous amounts of text, repeatedly tries to predict missing or next words, and back-propagation nudges every word's vector until predictions improve. Words used in similar company end up close together.

The dimensions have no names. Our toy axes ("action type", "tense") are labelled for teaching; in a real model meaning is spread across hundreds or thousands of unlabelled numbers.

Model	numbers per token
word2vec (2013)	300
BERT-base (2018)	768
GPT-2 small (2019)	768
GPT-3 (2020)	12 288

We can draw two or three dimensions; the mathematics works the same in 12 288. More dimensions leave room for finer shades of meaning.

word2vec and GloVe produce **static** embeddings: one fixed vector per word. That turns out not to be enough ...

The "Battery" Problem: One Word, Several Meanings

Same spelling, different meaning

- "The artillery **battery** fired at dawn." — a unit of guns
- "The radio **battery** died at dawn." — a power cell
- "The suspect was charged with **battery**." — an offence

A **static** embedding gives "battery" one point on the map — somewhere between guns and power cells. The model then has to guess which meaning is intended, and the next-word predictions blur.

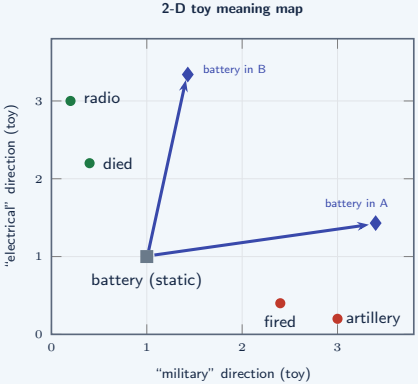
What the next word depends on

context so far	likely next words
the artillery battery ...	fired, displaced, commander
the radio battery ...	died, charged, pack

Goal: a **contextual** embedding — a vector for "battery" that has been *pulled* by its neighbours towards the right meaning, for this sentence only.

Producing contextual embeddings is the central job of every transformer layer.

Running Example: Context Pulls "Battery" Two Different Ways



Two sentences

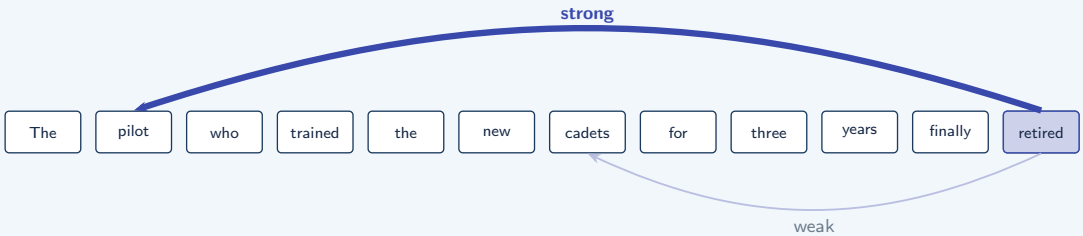
A: "artillery **battery** fired" **B**: "radio **battery** died"
Toy vectors: artillery $(3, 0.2)$, fired $(2.4, 0.4)$, radio $(0.2, 3)$, died $(0.4, 2.2)$, battery $(1, 1)$

Result of one attention step (next slides)

	sentence A	**sentence B**
attention output z	$(2.40,\ 0.43)$	$(0.43,\ 2.34)$
battery $+ z$	$(3.40,\ 1.43)$	$(1.43,\ 3.34)$

The *same* starting vector for "battery" moves towards the guns in A and towards the power cells in B. That moved vector is its **contextual embedding**.

Context Can Come From Far Away



Who retired?

The pilot — nine words earlier — not the nearer "cadets" or "years". Looking only at the last few words gets this wrong.

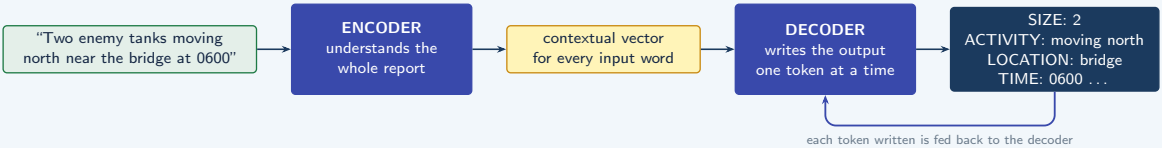
Clues can be paragraphs away

"... trained for months on the new 155 mm guns ... [three sentences later] ... at dawn the **battery** ____" — the deciding clue is far back.

Attention looks at every word, however distant, and weighs its relevance. Distance alone does not decide.

Older recurrent networks passed information word by word, so clues faded over long distances. Removing that bottleneck is why transformers handle long reports well.

Two Halves: The Encoder Reads, the Decoder Writes



Encoder

Reads the whole input at once and produces a contextual embedding for each token.

Decoder

Starts from a special *start* token; at each step uses (1) the encoder's vectors and (2) everything it has written so far.

Same pattern, many tasks

Free text $\rightarrow$ structured report (shown), translation, summarisation, question answering.

One sentence becomes many training examples

input x (words so far)	target y
Request	immediate
Request immediate	air
Request immediate air	support
Request immediate air support	for
Request immediate air support for	the
Request immediate air support for the	platoon

Self-supervised: no human labelling — the next word in real text *is* the label. Any manual, report or book becomes training data.

Training

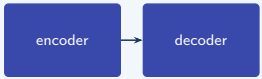
The untrained model guesses (say "drop" instead of "support"); the error is measured with cross-entropy and back-propagated. Across billions of examples this sets the embeddings, the attention matrices W^Q, W^K, W^V and every other weight.

Inference

The trained model is used on new text with its weights *frozen*. Everything later in this lecture describes what happens at inference.

Like a cadet: years of drills (training), then decisions in the field using what was learned (inference).

BERT, GPT and the Original Design



original Transformer, T5



BERT (encoder only)

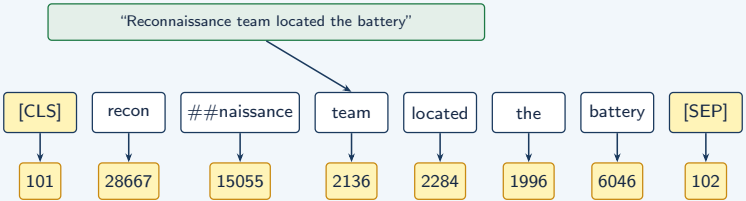


GPT family (decoder only)

	BERT-base	GPT-3
part used	encoder	decoder
numbers per token	768	12 288
layers (blocks)	12	96
attention heads	12	96
vocabulary	30 522	≈50 000
best at	understanding	generating

"Transformer" is the general design; BERT and GPT are particular models built from it. A GPT-style decoder still builds contextual embeddings — it simply uses only the words to its left.

Inside the Encoder, Step 1: Tokens and Token IDs



Tokens, not words

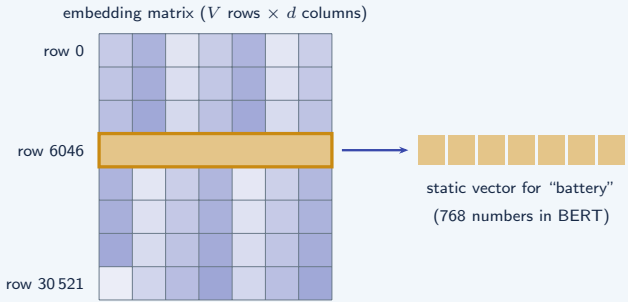
Common words are one token; long or rare words split into pieces ("##" marks a continuation in BERT's tokenizer). Treating tokens as words is fine for intuition.

Special tokens

BERT adds [CLS] at the start (a summary slot, used for classification) and [SEP] to mark the end of a sentence or the boundary between two.

Each token has an ID: its row number in the vocabulary (30 522 entries for BERT). [CLS] $= 101$ and [SEP] $= 102$ in BERT; the other IDs here are illustrative.

Step 2: Look Up Each Token's Vector



A lookup, not a calculation

The token ID is simply a row number. The model copies that row — the token's **static embedding**. The whole table was learned during training.

What training also learned

Besides this table, training fixed the attention matrices W^Q, W^K, W^V and all the other weights we meet next. At inference none of them change.

At this point "battery" has the *same* vector in every sentence. The next steps make it context-aware.

Step 3: Stamp Each Vector With Its Position

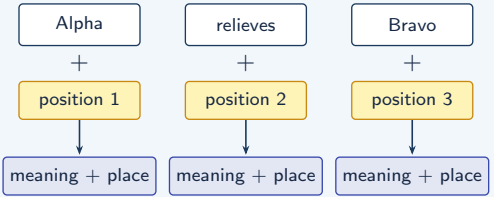
Order changes meaning

"**Alpha** relieves **Bravo**" vs "**Bravo** relieves **Alpha**"
Same words, opposite orders — and opposite consequences for who stays in the line.

A transformer processes all tokens **in parallel**, not one after another. Without extra information it would see these two sentences as the same bag of words.

FIX: ADD A POSITION VECTOR

$$\text{input}_t \;=\; \text{embedding}_t \;+\; \text{position}_t$$

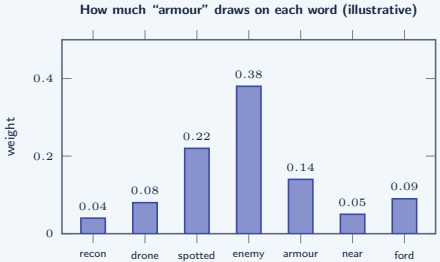


How the position vectors are made

The original paper used sine and cosine waves of many frequencies — like reading a clock's second, minute and hour hands together — so every position gets a unique, bounded code. Many later models learn the position vectors instead.

The formula and a worked example are in Lecture 9.

Step 4: Attention — Which Words Should Update Which?



Sentence: “recon drone spotted enemy armour near ford”

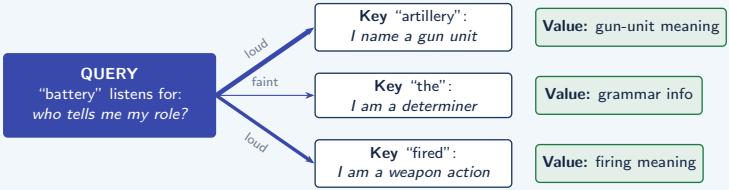
Reading the chart

“enemy” (0.38) tells us *whose* armour; “spotted” (0.22) says what happened to it; “armour” itself (0.14) keeps its own meaning. “recon” adds little to the meaning of “armour”. The weights sum to 1.

Replace “enemy” with “friendly” and the contextual vector for “armour” changes a lot; replace “near” with “by” and it barely moves. High weight = big influence.

Every word gets such a set of weights over every other word (and itself): an $n \times n$ table of attention weights.

Query, Key, Value: The Radio-Net Analogy



Query q

The call-sign a station is **listening for** — what this word needs to know.

Key k

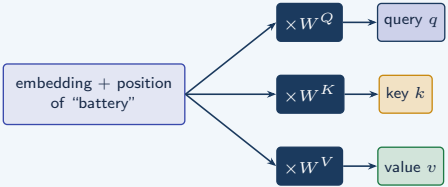
The call-sign each station **transmits** — what kind of information it offers. A good match $q \cdot k$ turns the volume up.

Value v

The **message** itself. The listener hears every message, each at its own volume, and sums them.

The volume mixer always adds up to 100% (softmax). The "call-signs" are not words like these in reality: they are learned vectors, and the model decides what to listen for.

Where Q, K and V Really Come From



THREE LEARNED MATRICES, APPLIED TO EVERY TOKEN

$$q = x W^Q, \qquad k = x W^K, \qquad v = x W^V$$

What each matrix "knows"

- W^Q: how to turn a token into a *question*
- W^K: how to turn a token into an *advertisement*
- W^V: how to turn a token into the *content* it hands over

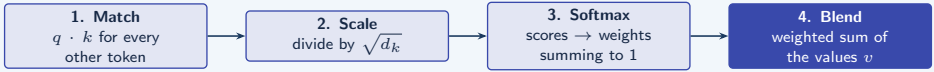
All three are learned in (pre-)training and then fixed.

Sizes per head

BERT-base: $768 \to 64$ numbers; GPT-3: $12\,288 \to 128$. Each token's q, k and v are much shorter than its embedding.

Every token is projected at the same time, so Q, K, V are matrices with one row per token.

The Recipe in Four Steps



ALL TOKENS AT ONCE

$$\text{Attention}(Q, K, V) \quad = \quad \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right) V$$

Why the transpose K^{T}?

For one query and one key, $q \cdot k$ is a single number. Stacking all keys as rows of K, the product QK^{T} computes every query–key match in one go: an $n \times n$ table.

Why divide by $\sqrt{d_k}$?

Dot products grow with vector length. Without scaling, softmax would put nearly all weight on one word and learning would stall. GPT-3: $d_k = 128$, divide by 11.3.

Worked Example: The Arithmetic Behind the "Battery" Picture

Simplest case for hand arithmetic: $W^Q = W^K = W^V = I$ (identity), so $q = k = v = x$; $d_k = 2$.

Sentence A: "artillery battery fired"

key	$q_{\text{battery}} \cdot k$	$\div\sqrt{2}$	$e^{(\cdot)}$	weight
artillery $(3, 0.2)$	3.2	2.263	9.609	**0.458**
battery $(1, 1)$	2.0	1.414	4.113	0.196
fired $(2.4, 0.4)$	2.8	1.980	7.242	0.345
			sum 20.965	1.000

Blend:

$z = 0.458(3, 0.2) + 0.196(1, 1) + 0.345(2.4, 0.4) = (\mathbf{2.40},\ \mathbf{0.43})$

Add to the original (residual): $(1, 1) + (2.40, 0.43) = (\mathbf{3.40},\ \mathbf{1.43})$ — deep in "military" territory.

Sentence B: "radio battery died"

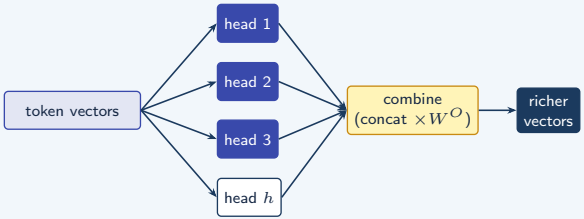
Scores $3.2,\ 2.0,\ 2.6 \rightarrow$ weights $0.480,\ 0.206,\ 0.314$

$$z = (0.43,\ 2.34), \qquad x + z = (1.43,\ 3.34)$$

Identical query, identical arithmetic — different neighbours, so a different result. That is all a contextual embedding is.

Real models use *learned* W^Q, W^K, W^V, not the identity: the question a word asks can differ from what it advertises and what it hands over. Lecture 9 works that case.

Multi-Head Attention: A Sensor Suite, Not a Single Sensor



Each head has its own W^Q, W^K, W^V

So each asks a different question of the same sentence, in parallel. Their outputs are joined and mixed by one more matrix W^O.

Analogy: a surveillance sensor suite

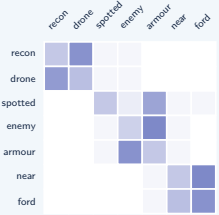
Radar, infrared and a camera watch the same scene. Each picks up what the others miss — range, heat, shape — and the fused picture beats any single sensor. Heads do the same for relationships between words.

Real numbers

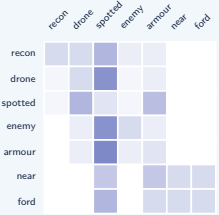
BERT-base: 12 heads $\times$ 64 $=$ 768. GPT-3: 96 heads $\times$ 128 $=$ 12 288. The head size is chosen so the heads together match the embedding width.

Nobody assigns roles to heads; specialisations emerge during training.

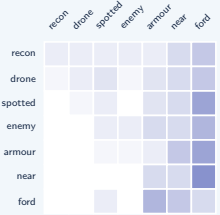
What Different Heads Might Track (Illustrative)



head A: modifiers



head B: action



head C: location

Modifier head

"enemy" ↔ "armour", "recon" ↔ "drone": which words describe which.

Action head

"spotted" connects *who* spotted (drone) with *what* was spotted (armour).

Location head

"near" and "ford" feed *where* into the other words — vital for a contact report.

Patterns like these have been observed in trained models, but real heads are messier and many overlap or do little. Treat heat-maps as clues to what a model attends to, not as proof of its reasoning.

Residual connection: margin notes, not a rewrite

The output of attention is *added* to the original vector:

$$x_{\text{new}} = x + \text{Attention}(x)$$

Like an analyst writing notes in the margin of a report instead of retyping it: the original survives, annotations accumulate. (We did exactly this for "battery": $(1, 1) + z$.)

Bonus: during training, gradients flow straight through the "+", so very deep stacks still learn.

Layer normalisation: bring every reading to a common scale

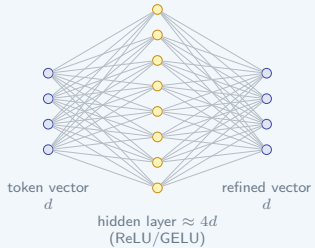
Each token's vector is shifted to mean 0 and scaled to standard deviation 1 — like calibrating instruments before fusing their readings.

Example: $(2, 4, 6, 8)$: mean 5, std 2.236

$$\rightarrow (-1.342,\ -0.447,\ 0.447,\ 1.342)$$

Normalisation keeps numbers in a stable range layer after layer, which makes training faster and more reliable. A learned scale and shift follow.

The Feed-Forward Network: Each Token Thinks It Over



POSITION-WISE FEED-FORWARD

$$\mathrm{FFN}(x) \;=\; W_2\, f(W_1 x + b_1) + b_2, \quad f \text{ non-linear}$$

Why another layer after attention?

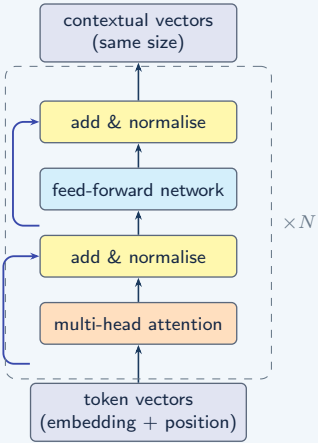
Attention *gathers* information from other words but is largely a weighted averaging step. The feed-forward network then *processes* what each token gathered, with a non-linear activation that can capture patterns attention alone cannot.

Briefing room, then desk work

Attention is the staff briefing where everyone hears from everyone. The FFN is each officer back at their desk working through their own notes — same procedure for every officer, done in parallel.

BERT-base: $768 \to 3\,072 \to 768$; GPT-3: $12\,288 \to 49\,152 \to 12\,288$. Most of a model's parameters live in these layers.

One Transformer Block — Then Stack It



Stacking blocks

Each block takes vectors in and gives same-sized vectors out, so blocks stack like layers of analysis: early blocks resolve local word meanings ("battery" = gun unit); later blocks build sentence- and paragraph-level understanding.

Model	blocks N
original Transformer (each side)	6
BERT-base / BERT-large	12 / 24
GPT-3	96

Parallel throughout: every token goes through every head and the feed-forward network at the same time — one reason transformers train fast on GPUs.

Turning a free-text report into a structured one

Input report (encoded once): "Two enemy tanks moving north near the bridge at 0600"

step	decoder has written so far	next token
1	⟨start⟩	SIZE
2	⟨start⟩ SIZE	2
3	⟨start⟩ SIZE 2	ACTIVITY
4	⟨start⟩ SIZE 2 ACTIVITY	moving
5	. . . ACTIVITY moving	north
. . .	. . .	⟨end⟩

Two inputs at every step

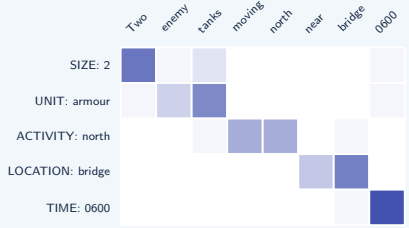
1. The encoder's contextual vectors for the whole report
2. The tokens the decoder has produced so far

Masked self-attention

While writing step 4, the decoder may look at steps 1–3 but not ahead. During training the correct output is supplied and a *mask* hides future tokens, so all steps can be trained in parallel.

At each step the output layer gives a probability for every vocabulary token; the most likely (or a sampled) one is written and fed back.

Cross-Attention: The Decoder Consults the Report



illustrative cross-attention weights

Q from one place, K and V from another

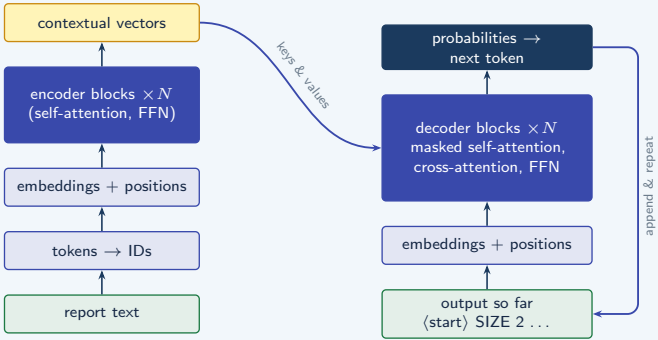
- **Query** — from the decoder: "I am writing the LOCATION field; which part of the report do I need?"
- **Keys and values** — from the encoder's vectors for the input report

Because Q and K, V come from *different* sequences, this is called **cross**-attention.

Each output field "looks up" the words that justify it: LOCATION draws on "near the bridge", TIME on "0600". Translation works the same way, aligning output words with input words.

Encoder-only BERT has no cross-attention; decoder-only GPT has none either — its prompt and its answer are one sequence.

The Whole Journey on One Slide



Everything on this slide is computed with the operations in this lecture: lookups, additions, matrix products, softmax and a small non-linear network — repeated at scale.

Defence Relevance and Cautions

Where the intuition pays off

- **Contextual meaning**: "battery", "fire", "contact" and "engage" change meaning with context — exactly what attention resolves
- **Structured reporting**: encoder–decoder models turning field notes into SALUTE, SITREP or 9-line formats for human checking
- **Search by meaning**: embeddings let analysts find reports about "armour" that never use the word "tank"

Cautions

- Fluent output is not verified output — models can invent details (hallucination)
- Jargon, abbreviations and grid references may be split into many tokens and mishandled
- Text inside an input document can try to steer the model (prompt injection)
- Attention maps suggest, but do not prove, why a model answered as it did

Keep a trained human in the loop for any output that feeds a decision.

Further Exploration

Interactive and video

- **Transformer Explainer** (Georgia Tech Polo Club) — runs a small GPT-2 in the browser and shows every step: poloclub.github.io/transformer-explainer
- **3Blue1Brown**, "Neural Networks" series — visual chapters on GPT, attention and embeddings
- **codebasics**, "Transformers Explained — Simple Explanation of Transformers" (2025) — an intuitive video walk-through of the same topics

In this course

- Lecture 6, Part I, *Why Hidden Layers?* — non-linearity and feature maps
- Lecture 6, Part II, *Backpropagation by Hand* — how all these weights are learned
- Lecture 9, *Transformers & Attention* — full mathematics, sinusoidal encoding, multi-head worked example and the Vaswani et al. figure block by block

Primary source

Vaswani, A. et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems 30.

Summary and Quick Check

LANGUAGE MODEL

predict the next token; repeat to write

EMBEDDING

a learned vector = coordinates on a meaning map

CONTEXTUAL EMBEDDING

static vector + what attention gathers from context

ATTENTION

$\text{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})\,V$ — listen, weigh, blend

BLOCK

multi-head attention → add &
norm → FFN → add & norm

Quick check

1. Why can't token ID numbers be used directly as meanings?
2. In sentence B, which word gave "battery" the most attention weight, and how much?
3. In the radio-net analogy, what plays the role of the key?
4. Why does the decoder need a mask during training?
5. In cross-attention, where do the queries come from, and where do the keys and values come from?

From intuition to the full mathematics

- Word vectors and word arithmetic: tiger − man + woman ≈ tigress
- Sinusoidal positional encoding, computed and explained
- Scaled dot-product attention $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ by hand
- A two-head multi-head example with Add & Norm
- Every block of Figure 1 of Vaswani et al. (2017)
- BERT vs GPT vs encoder–decoder; defence uses and cautions

Assignment 8: cosine similarity on a meaning map + a static-vs-contextual explanation + one attention step by hand + encoder/decoder roles and masking + a defence-use assessment; bonus: what two heads add.

Preview: "enemy tank advances"

Three tokens, four-dimensional toy vectors, every number checked: embeddings, positions, queries, keys and values, two heads, a residual and a layer norm — the whole encoder block on one sentence.

Today's analogies become equations next lecture: the radio net is $QK^\top$, tuning in is the softmax, and the message received is the weighted sum of V.